

Composite

Un patron structurel

Préparé par MAA S.Yahiouche

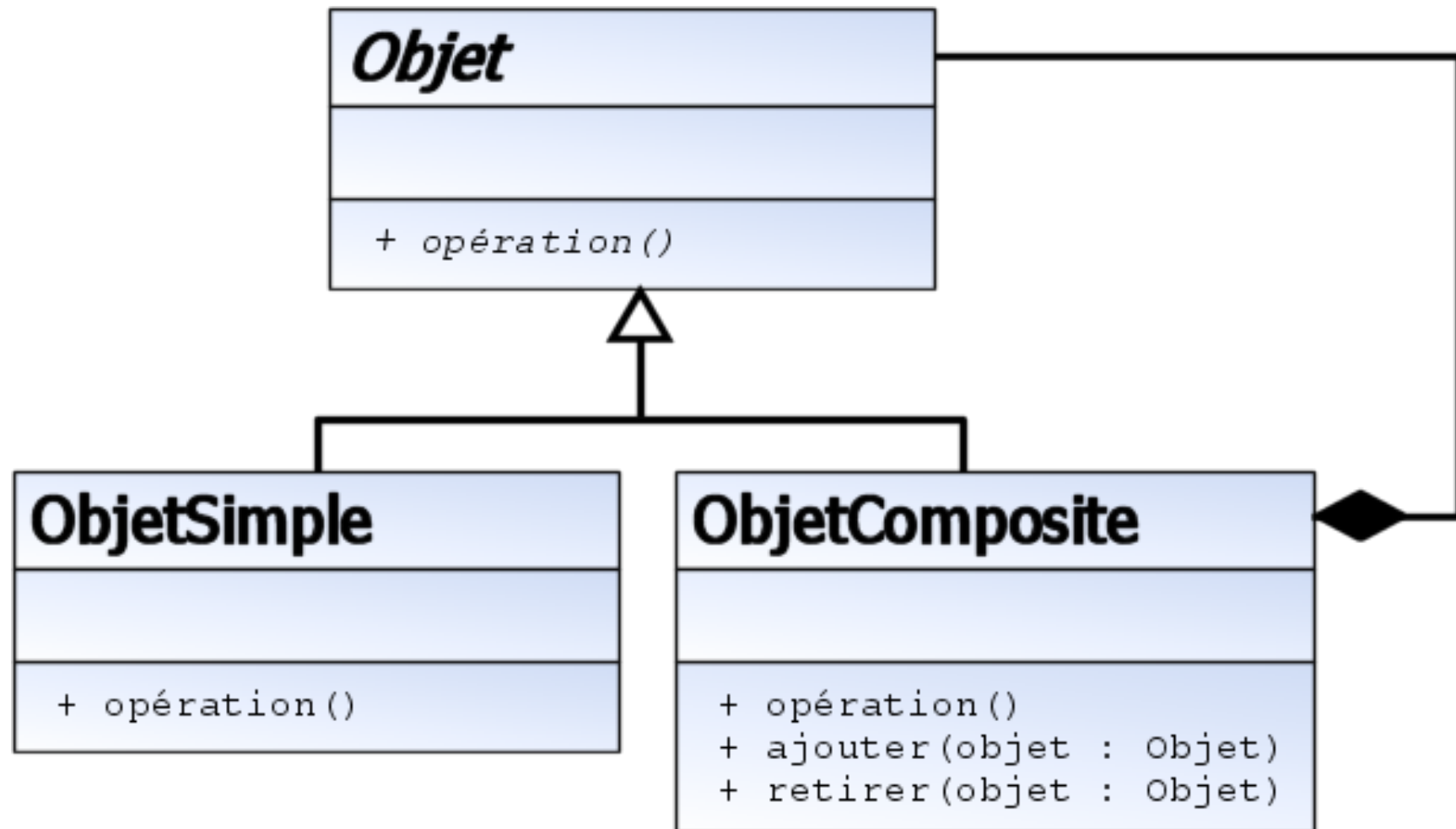
Composite: But

- Composer des objets dans des structures d'arbre pour représenter des hiérarchies composants/composés
- Composite permet au client de manipuler uniformément les objets simples et les objets au sein de leurs compositions

Exemples

- Structure hiérarchique d'une entreprise :
techniciens, cadres (employé)
- JComponent / JButton, JLabel, etc. ou
`java.awt.Component` / `java.awt.Container`
- Structures des dossiers et fichiers

Composite Diagramme UML



Les participants au diagramme:

- **Objet**
 - déclare l'interface pour la composition d'objets
 - met en œuvre le comportement par défaut
- **ObjetSimple**
 - représente les objets manipulés, ayant une interface commune
- **ObjetComposite**
 - définit un comportement pour les composants ayant des enfants
 - stocke les composants enfants
 - met en œuvre la gestion des composants enfants

- Dans ce patron, un **objet composite** est constitué d'un ou de plusieurs objets similaires (ayant des fonctionnalités similaires).
- L'idée est de manipuler un groupe d'objets de la même façon que s'il s'agissait d'un seul objet.
- Les objets ainsi regroupés doivent posséder des opérations communes, c'est-à-dire un "dénominateur commun".

- le patron de conception Composite consiste donc en la définition d'une classe abstraite possédant des opérations génériques à un groupe d'objets et qui permet la délégation d'opération aux objets eux-même par un appel en cascade.

L'interface composant

l'interface peut être remplacé par une classe abstraite s'il y a des comportements commun à tous les types d'objets voulant être implémentés

```
public Interface Composant {  
public void operation(); }
```

La classe composite

Pour représenter les noeuds pouvant avoir des Composants sous-hiérarchiquement.

```
public class Composite implements Composant {  
protected ArrayList<Composant> components = new  
    ArrayList<Composant>();  
@Override  
public void operation() { //À compléter }  
  
public void add(Composant c) {  
    components.add(c); }  
  
public void remove(Composant c) {  
    components.remove(c); }  
}
```

Classe Feuille ou Leaf

- Pour représenter des feuilles ou bien des noeuds ne pouvant avoir des Component sous-hiérarchiquement.

```
public class Leaf implements Composant  
{  
@Override  
public void operation() { //À compléter } }
```

Exemple vu en cours

Interface Component.java

- public interface Employee
- {
- public void showEmployeeDetails();
- }

Leaf.java

```
public class Developer implements Employee
{
    private String name;
    private long empld;
    private String position;

    public Developer(long empld, String name, String position)
    {
        this.empld = empld;
        this.name = name;
        this.position = position;
    }

    @Override
    public void showEmployeeDetails()
    {
        System.out.println(empld+" " +name+);
    }
}
```

Une deuxième feuille...

Leaf.java

```
public class Manager implements Employee
{
    private String name;
    private long empld;
    private String position;

    public Manager(long empld, String name, String position)
    {
        this.empld = empld;
        this.name = name;
        this.position = position;
    }

    @Override
    public void showEmployeeDetails()
    {
        System.out.println(empld+" " +name);
    }
}
```

Composite.java

```
import java.util.ArrayList;
import java.util.List;

public class CompanyDirectory implements Employee
{
    private List<Employee> employeeList =
        new ArrayList<Employee>();

    @Override
    public void showEmployeeDetails()
    {
        for(Employee emp:employeeList)
        {
            emp.showEmployeeDetails();
        }
    }

    public void addEmployee(Employee
        emp)
    {
        employeeList.add(emp);
    }

    public void removeEmployee(Employee
        emp)
    {
        employeeList.remove(emp);
    }
}
```

Client.java

```
public class Company
{
    public static void main (String[] args)
    {
        Developer dev1 = new Developer(100,
        »Latrech Shaima", "Pro Developer");
        Developer dev2 = new Developer(101,
        »Dob Said", "Developer");
        CompanyDirectory engDirectory =
        new CompanyDirectory();
        engDirectory.addEmployee(dev1);
        engDirectory.addEmployee(dev2);

        Manager man1 = new Manager(200,
        "Bil gates", "SEO Manager");
        Manager man2 = new Manager(201,
        "Vidal clowny ", »radia's Manager");
```

```
        CompanyDirectory accDirectory = new
        CompanyDirectory();
        accDirectory.addEmployee(man1);
        accDirectory.addEmployee(man2);
```

```
        CompanyDirectory directory = new
        CompanyDirectory();
        directory.addEmployee(engDirectory);
        directory.addEmployee(accDirectory);
        directory.showEmployeeDetails();
```

```
    }
}
```

Exemple: fichier et dossier(à faire en td

- l'affichage des noms de fichiers contenus dans des dossiers :
- Pour un fichier, on affiche ses informations.
- Pour un dossier, on affiche les informations des fichiers qu'il contient.
- Dans ce cas, le patron composite est tout à fait adapté :
- L'Objet est de façon générale ce qui peut être contenu dans un dossier : un fichier ou un dossier,
- L'ObjetSimple est un fichier, sa méthode affiche() affiche simplement le nom du fichier,
- L'ObjetComposite est un dossier, il contient des objets (c'est à dire des fichiers et des dossiers). Sa méthode affiche() parcourt l'ensemble des objets qu'il contient (fichier ou dossier) en appelant leur méthode affiche().